

Reviewer Pack — Minimal Rank of Schur-Weyl Tensor Product Representations vs...

Entry 8383796bbc60 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 22:23:55 UTC

Minimal Rank of Schur-Weyl Tensor Product Representations vs DPLL Proof Length

Entry ID: 8383796bbc60 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 22:23:55 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality and Hecke algebras **Field B** (complexity object): DPLL proof length for CNF formulas

Statement:

[‘For every 3-CNF formula F with n variables, let V be the vector space of all Schur-Weyl tensor product representations of its clause-indicator polynomial. The conjecture is that there exists a constant C such that the minimal rank of V , denoted as $\min_rank(V)$, is upper bounded by $C * \log(n)$. Moreover, for any instance F with n variables, the length of the shortest DPLL proof of F , denoted as $DPLL_length(F)$, satisfies $DPLL_length(F) = \Theta(\min_rank(V))$.’]

Rationale (proposer’s reasoning):

[‘Schur-Weyl duality and Hecke algebras provide a powerful framework for studying the structure of representations in algebraic combinatorics. By linking this theory to the minimal rank of the clause-indicator polynomial, we aim to uncover a new perspective on proof complexity. If true, this conjecture would demonstrate that the DPLL proof length is closely related to the representation-theoretic properties of polynomials.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f00570cb221c9814

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman’s rank correlation coefficient between $\min_rank(V)$ and $DPLL_length(F)$ across 30 random seeds is > 0.5 and statistically significant (p-value < 0.05).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - schur-weyl duality hecke algebras AND dpll proof length cnf formulas - minimal rank schur-weyl tensor product representations clause-indicator polynomial log(n) - dpll proof length cnf formulas related to minimal rank schur-weyl tensor products

Top relevant hits considered: - [http://arxiv.org/abs/2603.00603v2] Mirabolic Hecke algebras, Schur-Weyl duality and Frobenius character formulas - [http://arxiv.org/abs/2009.03007v2] Affine Hecke algebras and their representations - [http://arxiv.org/abs/2307.06677v1] Quantum Schur-Weyl duality and q-Frobenius formula related to Reflection Equation algebras - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1407.1593v2] A Constructive Algorithm for Decomposing a Tensor into a Finite Sum of Orthonormal Rank-1 Terms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            random.shuffle(literals)
            clause = ' or '.join(f'x{i+1}' if l == 1 else f'-x{i+1}' for l in literals)
            clauses.append(clause)
        return ' and '.join(clauses)

    def clause_indicator_polynomial(clauses):
        n = len(clauses[0].split(' or '))
```

```

poly = [0] * (2 ** n)
for clause in clauses:
    bits = [1 if 'x' + str(i+1) in clause else 0 for i in range(n)]
    index = sum(bit << i for i, bit in enumerate(reversed(bits)))
    poly[index] += 1
return poly

def schur_weyl_rank(poly):
    n = len(poly)
    rank = 0
    for i in range(2 ** (n - 1)):
        subpoly = [poly[j] if (i & (1 << j)) else 0 for j in range(n)]
        if sum(subpoly) != 0:
            rank += 1
    return rank

def dpll_length(clauses):
    stack = []
    assignment = [None] * len(clauses)

    def backtrack(level):
        if level == len(clauses):
            return True
        for literal in [-1, 1]:
            var = abs(int(clauses[level].split('x')[1]))
            if (assignment[var - 1] is None and literal != assignment[var - 1]):
                assignment[var - 1] = literal
                stack.append((level + 1, literal))
                if backtrack(level + 1):
                    return True
                stack.pop()
                assignment[var - 1] = None
        return False

    return len(stack) if backtrack(0) else float('inf')

n = random.randint(5, 40)
formula = generate_3cnf(n)
poly = clause_indicator_polynomial(formula.split(' and '))
min_rank = schur_weyl_rank(poly)
dpll_len = dpll_length(formula.split(' and '))

return {
    "metric_name": "min_rank(V)",
    "metric_value": min_rank,
    "instances_tested": 1,
    "conjecture_holds": False if dpll_len == float('inf') else True,
    "counterexample": "" if dpll_len != float('inf') else "DPLL_length(F) is infinite"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value)**2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"DPLL_length(F) is infinite\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4fa5db9.py", line 87, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4fa5db9.py", line 69, in run_trial
    formula = generate_3cnf(n)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4fa5db9.py", line 26, in generate_3cnf
    clause = ' or '.join(f'x{i+1}' if l == 1 else f'-x{i+1}' for l in literals)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4fa5db9.py", line 26, in <genexpr>
    clause = ' or '.join(f'x{i+1}' if l == 1 else f'-x{i+1}' for l in literals)
                                   ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to compute the required Spearman's rank correlation coefficient and p-value for the conjecture. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12793
2	propose	ollama_remote	glm4:latest	0	0	10917
3	preregistration	ollama_remote	glm4:latest	0	0	5357
4	novelty	ollama_remote	glm4:latest	0	0	4785
5	novelty	ollama_remote	glm4:latest	0	0	7205
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21365
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18249
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10003
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18681
10	judge	ollama_remote	glm4:latest	0	0	36963

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146319 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/8383796bbc60.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8383796bbc60.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8383796bbc60.tar.gz` (if generated)